

NAME-	PrabhuDayal
SAP ID-	590033396
BATCH-	16 B.Tech C.S.E

Programming In C-HDOC DAY_16

Problem

Write a menu-driven C program using **switch-case** to perform digit-based operations on a positive integer. Define **one separate user-defined function for each menu option**, *with void return type and with a simple function call (without call by value/reference)*. Each function should read the required input, process it, and display the result. Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases.

- Check whether its digits are in increasing order from left to right.
- Check whether its digits are in decreasing order from left to right.
- Find the largest digit and the smallest digit.
- Find the second largest digit and second smallest digit in the number.

ALGORITHM

Step 1: Start.

Step 2: Declare a character array num1[100] to store the number and an integer c to store the user's choice.

Step 3: Read the number as a string.

Step 4: Display the following choices:

1. Check whether digits are in increasing order.
2. Check whether digits are in decreasing order.
3. Find the smallest and largest digits.
4. Find the second smallest and second largest digits.

Step 5: Read the user's choice.

Step 6: Use a switch statement according to the choice.

If choice is 1:

- Find the length of the number.
- Compare every digit with the digit immediately after it.
- If $n[i] > n[i+1]$, the number is not in increasing order.
- Otherwise, the number is in increasing order.
- Display the result.

If choice is 2:

HDOC DAY 16 REPORT

- Find the length of the number.
- Compare every digit with the digit immediately after it.
- If $n[i] < n[i+1]$, the number is not in decreasing order.
- Otherwise, the number is in decreasing order.
- Display the result.

If choice is 3:

- Assume the first digit is both the smallest and largest.
- Traverse through all digits.
- If a digit is smaller than the current smallest, update the smallest digit.
- If a digit is larger than the current largest, update the largest digit.
- Display the smallest and largest digits.

If choice is 4:

- Assume the first digit is both the smallest and largest.
- Traverse through all digits to find the smallest and largest digits and their indexes.
- Find an index that is neither the smallest nor the largest index.
- Use that digit as the initial value for the second smallest and second largest.
- Traverse through the digits again, skipping the smallest and largest indexes.
- If a digit is smaller than secsam, update secsam.
- If a digit is larger than seclar, update seclar.
- Display the second smallest and second largest digits.

Step 7: If the choice is not 1, 2, 3, or 4, display "**Invalid choice.**"

Step 8: Stop.

C PROGRAM CODE

```
#include <stdio.h>
#include <string.h>
void isincreasing(char n[100])
{
    int len=strlen(n);
    int index=0;
    for(int i=0;i<len-1;i++)
    {
        if(n[i]>n[i+1])
        {
            index=1;
        }
    }
}
```

HDOC DAY 16 REPORT

```
        break;
    }
}
if(index==0)
{
    printf("The number is in increasing order.\n");
}
else
{
    printf("The number is not in increasing order.\n");
}
}
void isdecreasing(char n[100])
{
    int len=strlen(n);
    int index=0;
    for(int i=0;i<len-1;i++)
    {
        if(n[i]<n[i+1])
        {
            index=1;
            break;
        }
    }
    if(index==0)
    {
        printf("The number is in decreasing order.\n");
    }
    else
    {
        printf("The number is not in decreasing order.\n");
    }
}
void islarsam(char n[100])
{

```

HDOC DAY 16 REPORT

```
int len=strlen(n);
char sam=n[0];
char lar=n[0];
for(int i=0;i<len;i++)
{
    if(n[i]<sam)
        sam=n[i];
    if(n[i]>lar)
        lar=n[i];
}
printf("The smallest digit is %c and the largest digit is %c.\n", sam, lar);
}

void isseclarsam(char n[100])
{
    int len=strlen(n);
    char secsam;
    int large=0,small=0;
    char seclar;
    char sam=n[0];
    char lar=n[0];
    for(int i=0;i<len;i++)
    {
        if(n[i]<sam)
        {
            sam=n[i];
            small=i;
        }
        if(n[i]>lar)
        {
            lar=n[i];
            large=i;
        }
    }
    int ind=0;
    while(ind==small || ind==large)
```

HDOC DAY 16 REPORT

```
ind++;
seclar=n[ind];
secsam=n[ind];
for(int i=0;i<len;i++)
{
    if(i==small || i==large)
        continue;
    if(n[i]<secsam)
        secsam=n[i];
    if(n[i]>seclar)
        seclar=n[i];
}
printf("The second smallest digit is %c and the second largest digit is %c.\n", secsam, seclar);
}
```

```
int main()
{
    char num1[100];int c;
    printf("enter a num\n");
    scanf("%s", num1);
    printf("choice 1 is to check if the number is in increasing order of its digits from left to right\n");
    printf("choice 2 is to check if the number is in decreasing order of its digits from left to right\n");
    printf("choice 3 is to find the smallest and the largest digits\n");
    printf("choice 4 is to find the second smallest and the second largest digits\n");
    printf("enter choice\n");
    scanf("%d", &c);
    switch(c)
    {
        case 1:
        {
            isincreasing(num1);
            break;
        }
        case 2:
        {
```

HDOC DAY 16 REPORT

```
    isdecreasing(num1);
    break;
}
case 3:{
    islarsam(num1);
    break;
}
case 4:
{
    isseclarsam(num1);

    break;
}
default:
    printf("Invalid choice.\n");
    break;
}
return 0;
}
```